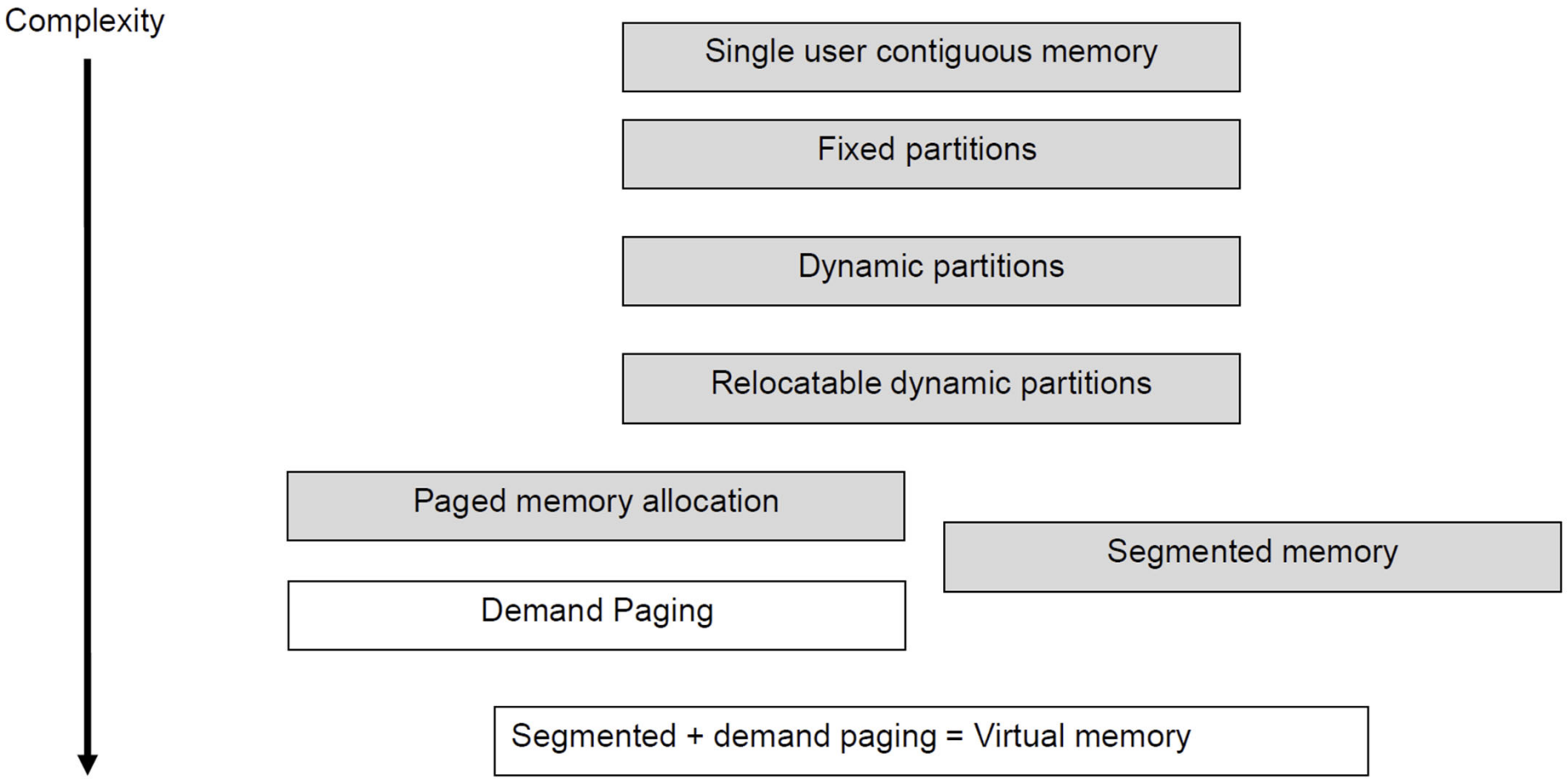
UNIVERSITY of York

System and Devices (SYS 3)

Lecture 13: Virtual Memory

Dr Pengcheng Liu

Recap


Complexity
Single user contiguous memory
Fixed partitions
Dynamic partitions
Relocatable dynamic partitions
Paged memory allocation
Segmented memory
Demand Paging
Segmented + demand paging = Virtual memory

Recap

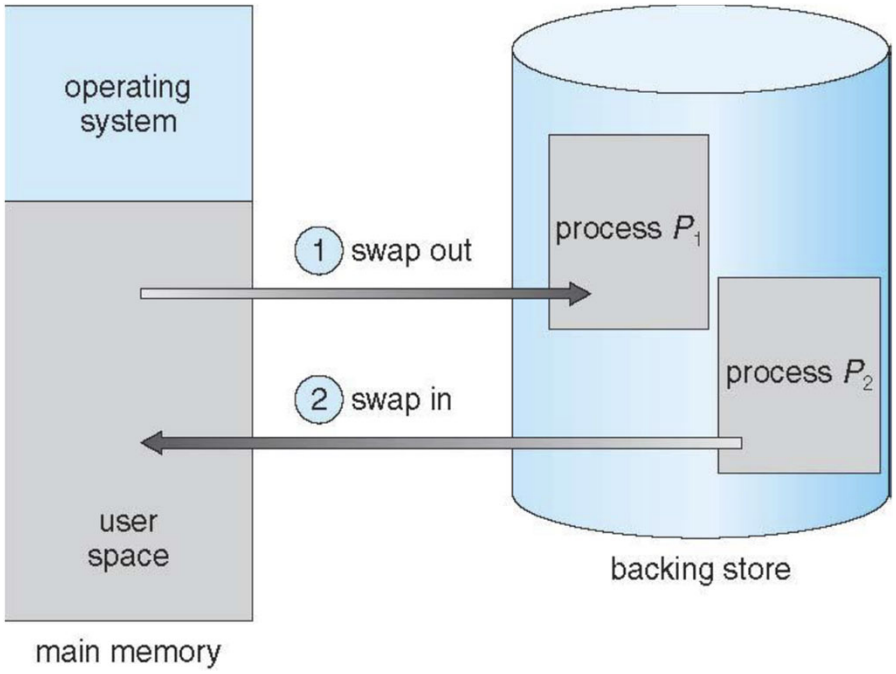
- A process can be swapped temporarily out of memory to a backing store, and then brought back into memory for continued execution

  - Total physical memory space of processes can exceed physical memory

- Backing store–fast disk large enough to accommodate binaries of all processes

- Major part of swap time is transfer time

  - Total transfer time is directly proportional to the amount of memory swapped



What are we going to learn today?

- Demand Paging and Virtual Memory

- Demand Paging Mechanism
  - Performance
  - Optimisation

Demand Paging

- Program is permanently stored in a backing store and is swapped in as needed
- Backing store is also split into storage units (called blocks), which are the same size as the frame and pages
- Program ends up with a small set of pages loaded in main memory – "working set"

  - Programs are executed (more or less) sequentially

  - Coverage of the program is generally small

    •many functions are seldom used: error handling routines, mutually exclusive modules, maintenance, etc.

Demand Paging

- **Advantages**

  - Allows to run programs which require much more memory than physically available (limited by secondary storage or addressing space)

- **Requirements**

  - Fast secondary storage device: DMA

  - More decisions to be taken by the OS:

    - what when all memory is full and a new page is needed?

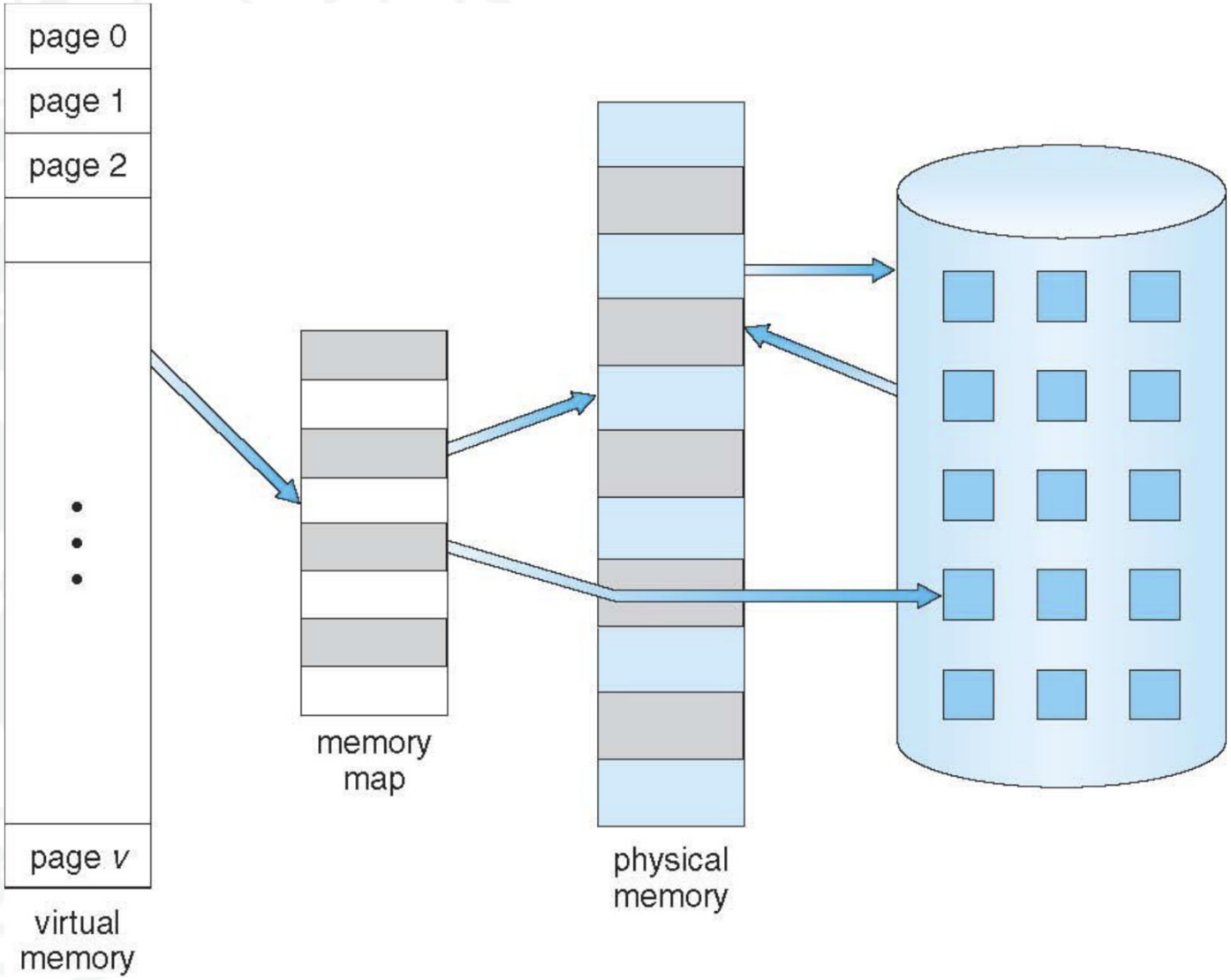
    - needed decision about which page is replaced

- Key enabler of virtual memory

Virtual Memory

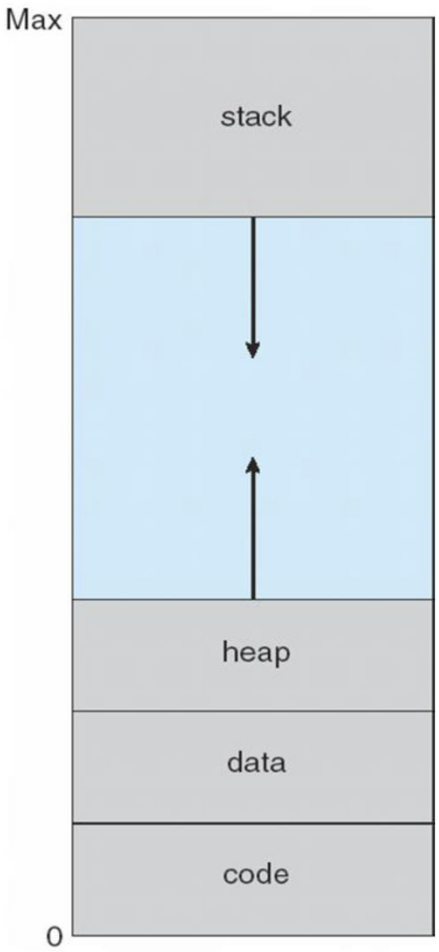
- Separation of user logical memory from physical memory
- Logical address space can be much larger than physical address space
  - Only part of the program needs to be in memory for execution
- More performance and resource efficiency
  - Less I/O needed to load or swap processes
  - Allows for more efficient process creation

Virtual Memory

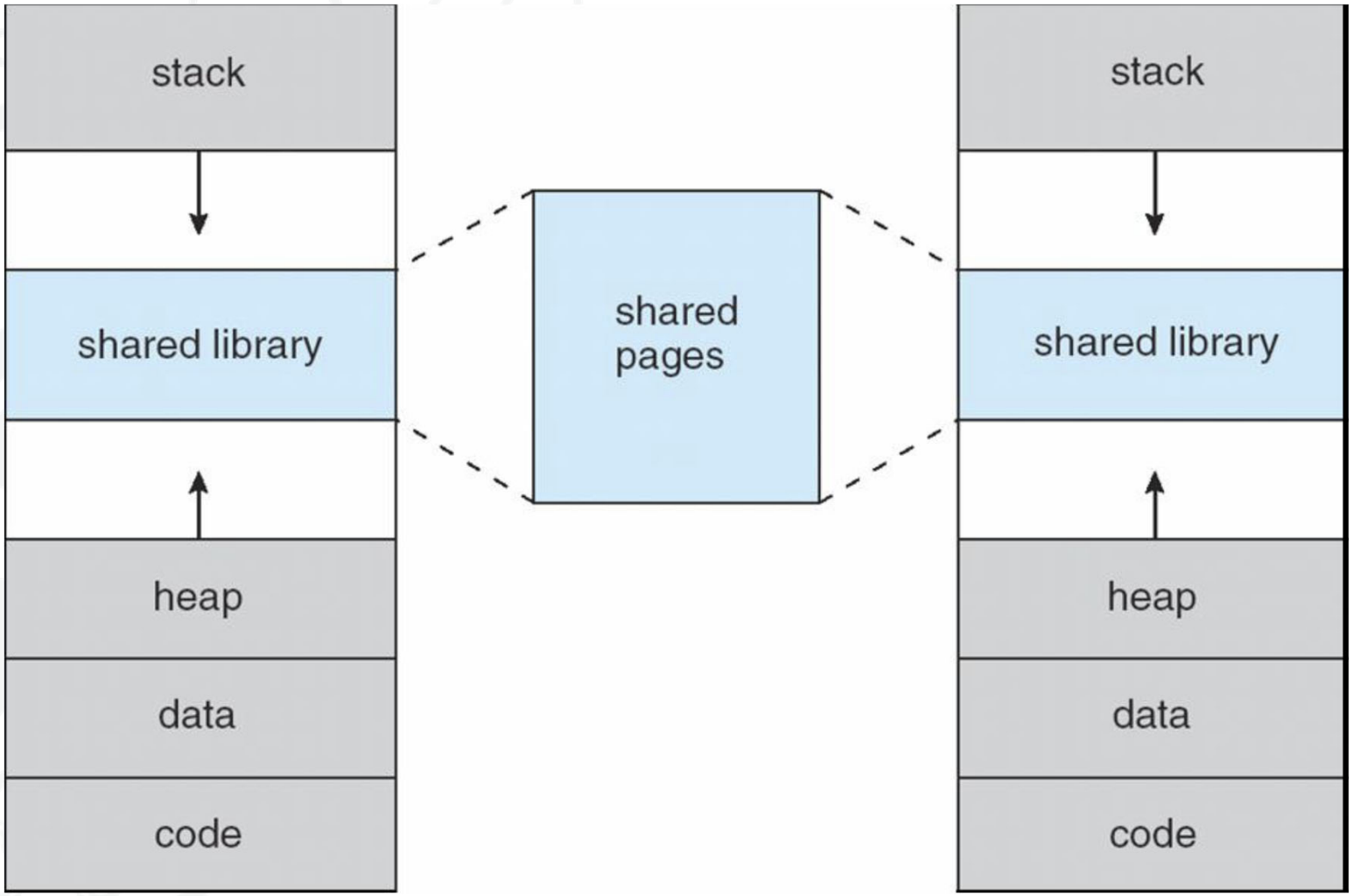


Virtual Address Space

- Usually design logical address space for stack to start at Max logical address and grow “down” while heap grows “up”
  - Maximizes address space use
  - Unused address space between the two
    - no physical memory needed until heap or stack grows to a given new page
- Enables sparse address spaces with holes left for growth, dynamically linked libraries, etc.
- System libraries shared via mapping into virtual address space
- Shared memory by mapping pages read-write into virtual address space
- Pages can be shared during fork(), speeding process creation

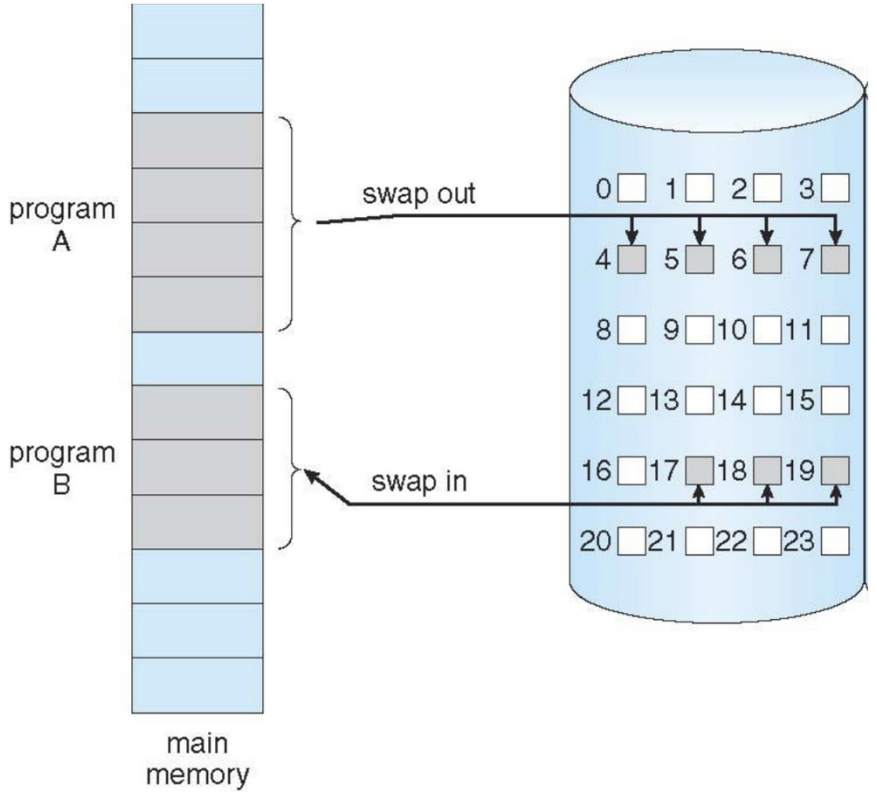


Shared Library Using Virtual Memory



Demand Paging Mechanism

- Similar to paging system with swapping
  - Pager: a swapper that deals with pages
  - Only loads needed pages
- Page is needed when there is a reference to it (read/write to its address range)
  - Invalid reference → abort
  - Not-in-memory →bring to memory



Demand Paging Mechanism

- When a process is to be swapped in, the pager predicts which pages will be used before the process is swapped out again

- Instead of swapping in a whole process, the pager brings into memory only its estimate of the working set

- OS must distinguish between the pages that are in memory and the pages that are on the disk

  - Uses a valid-invalid scheme

- If pages needed are already memory resident

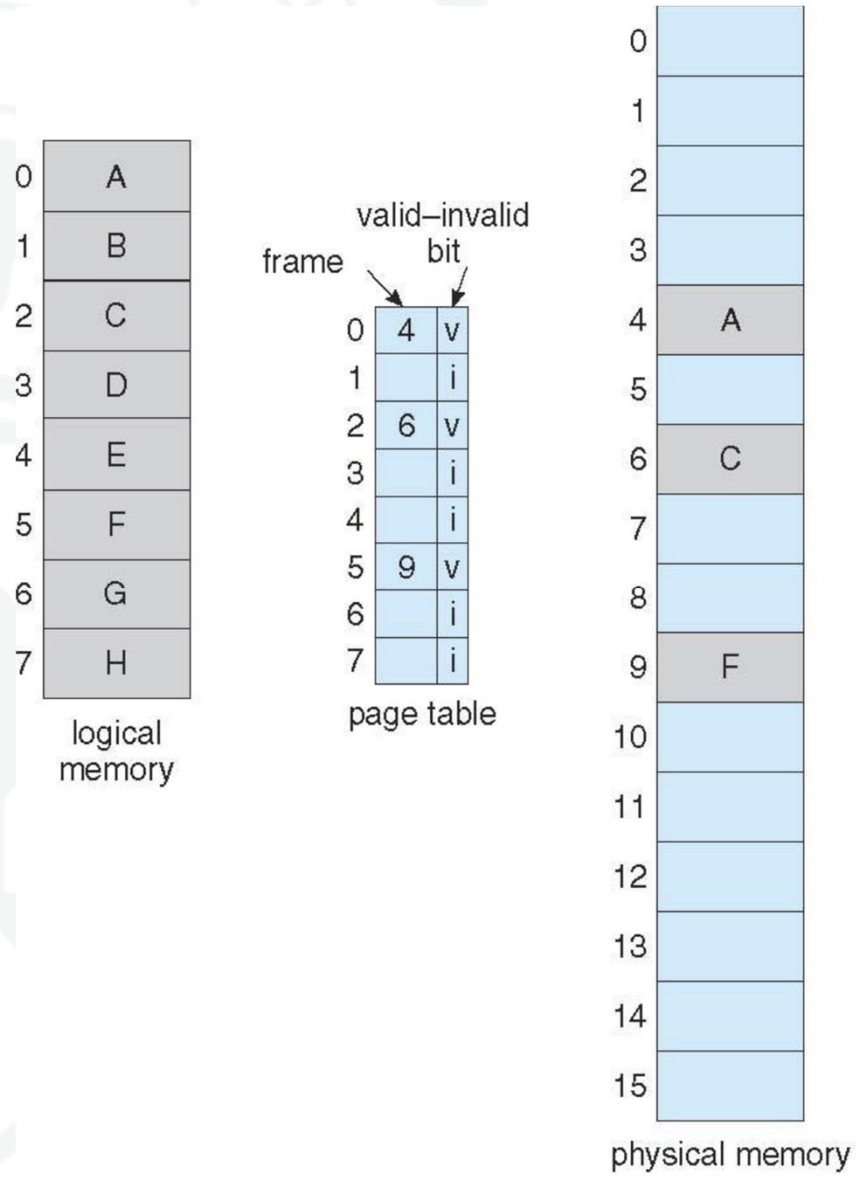
  - No difference from regular paging

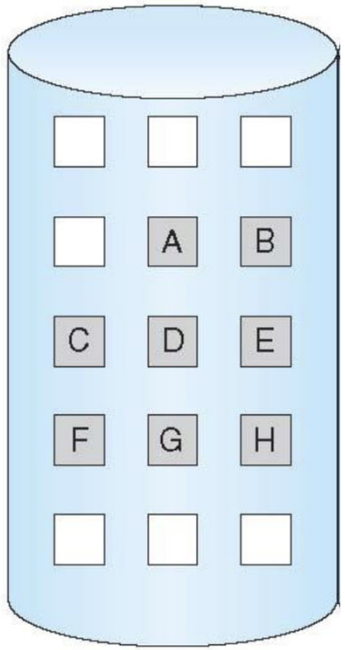
- If page needed and not memory resident→ page fault

  - Need to detect and load the page into memory from storage

    - Without changing program behavior
    - Without programmer needing to change code

Demand Paging Mechanism





Handling Page Faults

- Page fault is an interrupt -context switch ensues

  - Process state is saved

  - Enables OS to restart the instruction that caused the page fault, as the CPU will be in exactly the same state as prior to the memory reference

- OS actions upon page fault

  - Find free frame

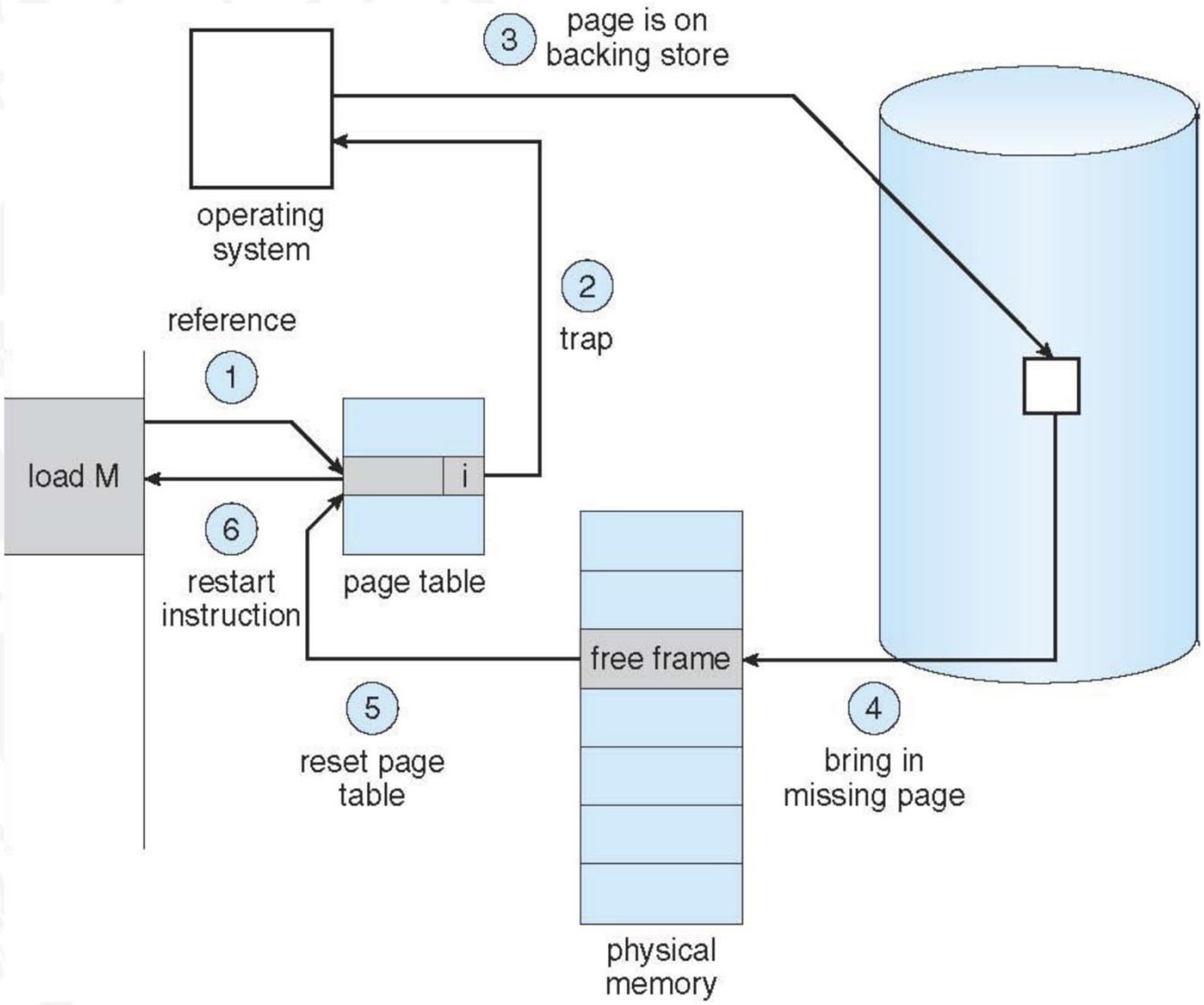
  - Swap page into frame via scheduled I/O operation

  - Reset tables to indicate page now in memory

    - set validation bit = v

  - Restart the instruction that caused the page fault

Handling Page Faults



Demand Paging Mechanism

- In the extreme case, a process may be started with none of its pages in memory

  - Pure demand paging

  - OS sets instruction-pointer to the first instruction of the process, non-memory-resident

    - page fault

  - Same for every other process pages on first access

- A single instruction can access multiple pages and cause multiple page faults

  - e.g. fetch and decode of instruction which adds 2 numbers from memory and stores result back to memory

  - The two numbers may reside in two different pages

Demand Paging Mechanism - Performance

- Worst case sequence of events

1. Trap to the OS kernel
2. Save the user registers and process state
3. Determine that the interrupt was a page fault
4. Check that the page reference was legal and determine location of page on disk
5. Issue a read from the disk to a free frame
    1. Wait in a queue for this device until the read request is serviced
    2. Wait for the device seek and/or latency time
    3. Begin the transfer of the page to a free frame
6. While waiting, allocate the CPU to some other process
7. Receive an interrupt from the disk I/O subsystem (I/O completed)
8. Save the registers and process state for the other process
9. Determine that the interrupt was from the disk
10. Correct the page table and other tables to show page is now in memory
11. Wait for the CPU to be allocated to this process again
12. Restore user registers, process state, new page table then resume interrupted instruction

Demand Paging Mechanism - Performance

- Three major activities
  - Service the interrupt: few hundred instructions
  - Read in the page: lots of time
  - Restart the process: few hundred instructions
- Page Fault Rate $0 \leq p \leq 1$
  - if p = 0 no page faults
  - if p = 1, every reference is a fault
- Effective Access Time (EAT)

EAT = (1 –p) x memory access time
+ p (page fault overhead
+ swap page out
+ swap page in)

Demand Paging Mechanism - Performance

- Example

  - Memory access time = 200 nanoseconds

  - Average page-fault service time = 8 milliseconds

  - EAT = (1 –p) x 200 + p x 8000000

    = 200 + p x 7999800

  - If one access out of 1,000 causes a page fault, then EAT = 8.2 microseconds-slowdown by a factor of 40!!

  - If want performance degradation < 10 percent

    220 > 200 + 7999800 x p

    20 > 7999800 x p

    p < 0.0000025

  - < one page fault in every 400,000 memory accesses

Demand Paging Mechanism - Optimisation

- Swap space I/O faster than file system I/O even if on the same device

  - swap allocated in larger chunks, less management needed than file system

- Copy entire process image to swap space at process load time

  - then page in and out of swap space

  - used in older BSD Unix

- Demand page in from program binary on disk, but discard rather than paging out when freeing frame

  - used in Solaris and current BSD

  - still need to write to swap space

    - pages not associated with a file (like stack and heap)
    - pages modified in memory but not yet written back to the file system

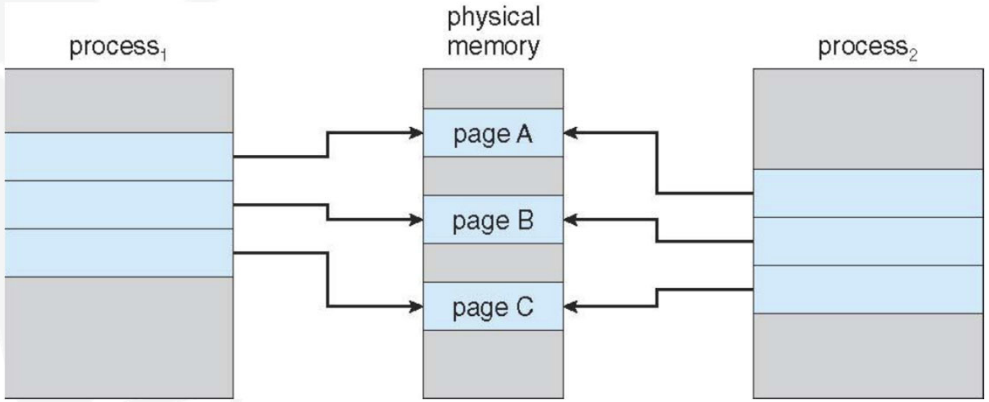
Demand Paging Mechanism - Optimisation

- Copy-on-Write (COW) allows both parent and child processes to initially share the same pages in memory
  - if either process modifies a shared page, only then is the page copied
- COW allows more efficient process creation as only modified pages are copied
- Variation on fork() system call has parent suspend and child using address space of parent
  - vfork() in Linux
  - Designed to have child call exec()
  - Very efficient

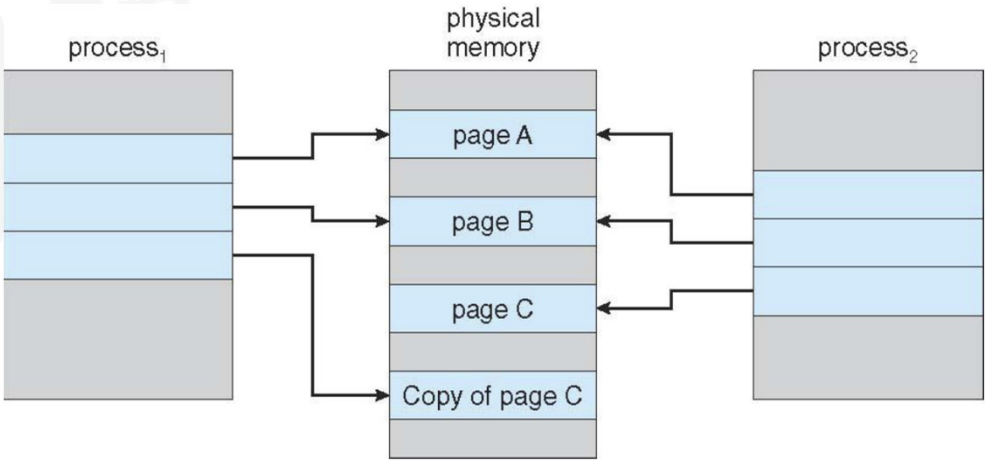
Demand Paging Mechanism - Optimisation

- Copy-on-Write (COW)

Before Process1 modifies Page C



After Process1 modifies Page C



Summary

- Virtual memory allows separation of logical and physical address spaces
  - Only part of the program must be loaded
  - Logical address space partitioned in fixed-sized pages
  - Memory partitioned in fixed-sized frames
- Demand paging loads fixed-sized blocks from backing store into memory frames when needed
  - if no free frames available, page replacement needed
  - replacement policies are crucial to system performance

Suggested Reading

- Silberschatz, chapter 10, 10th Edition